A Provably Terminating Sorting Algorithm With Unprovable Runtime

Kurt Gödel[1,†,*] Paul Erdős[2,†,*] Robin Young[3,*,✉]

[1]Institute for Advanced Study, Princeton NJ, USA
[2]Hungarian Academy of Sciences, Budapest, Hungary
[3]University of Cambridge, Cambridge, UK

[†]Contributed somewhat posthumously
[*]Equal contribution in the sense that all authors contributed equally to being on this paper
[✉]Corresponding author: robin.young@cl.cam.ac.uk

SIGBOVIK 2026

Abstract

We present GödelSort, a comparison-based sorting algorithm that correctly sorts any input array and provably terminates, yet whose termination cannot be proven in Peano Arithmetic (PA). Our algorithm achieves a runtime of $f_{\varepsilon_0}(n)$ in the fast-growing hierarchy, which dominates all primitive recursive functions. This is optimal among sorting algorithms whose termination proof requires transfinite induction up to ε_0. We provide a complete implementation and benchmark results demonstrating successful sorting for $n \leq 3$. For $n = 4$, our benchmarks are ongoing and expected to complete though we cannot prove this in PA. Our work opens new directions in pessimal algorithm design and demonstrates that the gap between "provably terminates" and "terminates in reasonable time" can be made arbitrarily large and indeed larger than any primitive recursive function.

1 Introduction

The analysis of sorting algorithms traditionally focuses on minimizing runtime. Quicksort [5] achieves $O(n \log n)$ expected time; heapsort [8] guarantees $O(n \log n)$ worst-case; radix sort [7] achieves $O(n)$ for bounded integers. These are considered "good" algorithms.

We take a different approach.

In this paper, we ask: *what is the slowest correct sorting algorithm?* More precisely, can we construct a sorting algorithm that:

1. Correctly sorts any input array
2. Provably terminates (in some formal system)
3. Has termination *unprovable* in Peano Arithmetic
4. Achieves runtime that grows faster than any primitive recursive function

We answer affirmatively by presenting GödelSort, which embeds a Goodstein sequence into the sorting procedure. Our main results are:

Theorem 1 (Main Result)**.** GödelSort *correctly sorts any array of n comparable elements in $\Theta(f_{\varepsilon_0}(n))$ time, where f_{ε_0} is the ε_0-level function in the fast-growing hierarchy. The termination of* GödelSort *is provable in ZFC but not in PA.*

Theorem 2 (Optimality)**.** *Among all sorting algorithms whose termination requires transfinite induction up to ε_0,* GödelSort *is asymptotically optimal so far.*

The second theorem follows trivially from the fact that we have not tried to optimize anything.

1.1 Motivation

Why would anyone want such an algorithm? We offer several justifications.

The independence of Goodstein's theorem from PA is a profound result in mathematical logic, but it is often presented abstractly. GödelSort provides a concrete computational artifact with code you can run, that terminates, but whose termination PA cannot prove.

Complexity theory typically studies the minimum resources required to solve a problem. The study of *maximum* resources (pessimal complexity) is equally valid and considerably less crowded.

It's funny.

1.2 Related Work

The study of intentionally slow algorithms has a rich if sparsely populated history. Bogosort [4] achieves $O(n!)$ expected runtime by random shuffling. Slowsort [1] achieves $\Omega(n^{\log n})$ via pessimal divide-and-conquer.

Our work differs fundamentally. GödelSort is not merely slow but *unprovably terminating* in PA. This places it in a different complexity class entirely as it intersects with mathematical logic rather than merely combinatorics.

Goodstein sequences were introduced by Goodstein [3] and shown to be independent of PA by Kirby and Paris [6]. To our knowledge, we are the first to weaponize this result for sorting.

2 Preliminaries

2.1 Hereditary Base Notation

Definition 3 (Hereditary Base-b Notation)**.** *A natural number n is written in hereditary base-b notation by expressing n in base b, then recursively expressing all exponents in base b, and so on until all numbers in the representation are less than b.*

Example 4. *The number* 266 *in hereditary base* 2*:*

$$\begin{aligned} 266 &= 256 + 8 + 2 \\ &= 2^8 + 2^3 + 2^1 \\ &= 2^{2^3} + 2^{2+1} + 2^1 \\ &= 2^{2^{2+1}} + 2^{2+1} + 2 \end{aligned}$$

All exponents are now in hereditary base 2*.*

2.2 Goodstein Sequences

Definition 5 (Goodstein Sequence)**.** *Given a starting value n, the Goodstein sequence G_n is defined as follows:*

1. *Write n in hereditary base 2*
2. *Replace all 2s with 3s ("bump" the base)*
3. *Subtract 1*
4. *Repeat with base $3 \to 4$, then $4 \to 5$, etc.*

Continue until (if?) the sequence reaches 0.

Example 6. *The Goodstein sequence starting from 3:*

$$\begin{aligned}
&G_3(1) = 3 = 2^1 + 1 \text{ in base } 2 \\
&G_3(2) = 3^1 + 1 - 1 = 3 \text{ (bump to base 3, subtract 1)} \\
&G_3(3) = 4^1 - 1 = 3 \text{ (bump to base 4, subtract 1)} \\
&G_3(4) = 5^1 - 1 - 1 = 2 \\
&G_3(5) = 1 \\
&G_3(6) = 0 \quad \checkmark
\end{aligned}$$

The remarkable fact is:

Theorem 7 ([3])**.** *Every Goodstein sequence eventually reaches 0.*

Theorem 8 ([6])**.** *Goodstein's theorem is unprovable in Peano Arithmetic.*

The proof of Goodstein's theorem requires transfinite induction up to $\varepsilon_0 = \omega^{\omega^{\omega^{\cdot^{\cdot^{\cdot}}}}}$, the first ordinal α satisfying $\omega^\alpha = \alpha$.

2.3 The Fast-Growing Hierarchy

Definition 9 (Fast-Growing Hierarchy)**.** *The fast-growing hierarchy $\{f_\alpha\}$ is defined for ordinals α by:*

$$\begin{aligned}
f_0(n) &= n + 1 \\
f_{\alpha+1}(n) &= f_\alpha^{(n)}(n) = \underbrace{f_\alpha(f_\alpha(\cdots f_\alpha}_{n \text{ times}}(n)\cdots)) \\
f_\lambda(n) &= f_{\lambda[n]}(n) \text{ for limit ordinals } \lambda
\end{aligned}$$

where $\lambda[n]$ is the n-th element of the fundamental sequence for λ.

For context:

- $f_1(n) = 2n$
- $f_2(n) = n \cdot 2^n$
- $f_3(n) \approx 2 \uparrow\uparrow n$ (tower of exponentials)
- $f_\omega(n) = f_n(n)$ (diagonalization)
- $f_{\varepsilon_0}(n) =$ *quite large*

The length of the Goodstein sequence starting from n is approximately $f_{\varepsilon_0}(n)$.

3 The Algorithm

We now present GödelSort in its full specifications.

Algorithm 1 GödelSort

Require: Array A of n comparable elements
Ensure: A sorted in non-decreasing order

```
1: m ← n
2: b ← 2
3: while m > 0 do                                   ▷ Goodstein sequence
4:     Write m in hereditary base b
5:     Replace all occurrences of b with b + 1
6:     m ← m − 1
7:     b ← b + 1
8: end while
9: return InsertionSort(A)                          ▷ Any correct sort suffices
```

Theorem 10 (Correctness)**.** GödelSort *correctly sorts any input array.*

Proof. Line 9 calls a correct sorting algorithm. □ □

Theorem 11 (Termination)**.** GödelSort *terminates on all inputs.*

Proof. The while loop implements a Goodstein sequence starting from n. By Goodstein's theorem, this reaches 0 in finite time. The proof requires transfinite induction up to ε_0 and cannot be carried out in PA. □ □

Theorem 12 (Runtime)**.** GödelSort *runs in time* $\Theta(f_{\varepsilon_0}(n))$*.*

Proof. The Goodstein sequence starting from n has length $\Theta(f_{\varepsilon_0}(n))$. The final insertion sort takes $O(n^2)$ time, which is negligible. □ □

4 Empirical Evaluation

We implemented GödelSort in Python and conducted extensive benchmarks.

n	Input	Goodstein Steps	Time	Status
1	$[1]$	1	0.000005s	✓
2	$[2, 1]$	3	0.000007s	✓
3	$[3, 1, 2]$	5	0.000011s	✓
4	$[4, 2, 3, 1]$	$\sim 3 \times 2^{402653211}$	—	Running
5	$[5, 3, 1, 4, 2]$	$> f_{\varepsilon_0}(5)$	—	Running

Table 1: Benchmark results for GödelSort. For $n \geq 4$, benchmarks are ongoing.

Remark 13. *The benchmark for $n = 4$ was started on Feb 4, 2026. At time of writing, after 10^6 iterations, m has grown to approximately 10^{11} and shows no signs of decreasing. We remain confident in eventual termination, though we cannot prove this confidence is warranted within PA.*

Remark 14. *For $n = 4$, the number of Goodstein steps is known to be exactly:*

$$G(4) = 3 \cdot 2^{402653211} - 2$$

At one step per nanosecond, this would require approximately $10^{121289586}$ years. The universe is approximately 1.4×10^{10} years old. We have applied for a compute grant.

4.1 Comparative Analysis

Algorithm	Runtime	PA Proves Termination?
Quicksort	$O(n \log n)$ expected	Yes
Heapsort	$O(n \log n)$	Yes
Bogosort	$O(n!)$ expected	Yes
Slowsort	$\Omega(n^{\log n})$	Yes
GödelSort	$\Theta(f_{\varepsilon_0}(n))$	**No**

Table 2: Comparison with existing sorting algorithms. GödelSort is the only algorithm in a complexity class that PA cannot reason about.

5 Extensions and Future Work

5.1 GödelSort++

One might ask: can we do worse? The answer is yes. By using the Hydra game of Kirby and Paris, or the Paris-Harrington theorem, we can construct sorting algorithms whose termination requires induction beyond ε_0.

Definition 15 (GödelSort++)**.** *Replace the Goodstein sequence with a Hydra battle. Each comparison spawns two new heads. The algorithm terminates when the Hydra is slain.*

We leave the implementation to future work, as our graduate students have expressed reluctance to wait $f_{\varepsilon_0+1}(n)$ steps for their code to compile.

5.2 Practical Applications

We anticipate GödelSort will find applications in:

- **Cryptography**: Any adversary attempting to brute-force a GödelSort-based system will face runtime that cannot even be proven finite in PA.
- **Job security**: Code that provably terminates but cannot be proven to terminate in any reasonable time ensures continued employment.
- **Philosophy**: When asked "will this code ever finish?" one can truthfully answer "Yes, but I cannot prove it."

5.3 Open Problems

1. Is there a sorting algorithm whose termination is independent of ZFC?
2. Can GödelSort be parallelized? (We conjecture no, since waiting cannot be distributed.)
3. What is the fastest algorithm whose termination is unprovable in PA? We have established an upper bound; lower bounds remain open.
4. What is the worst algorithm whose correctness requires univalence?

6 Conclusion

We have presented GödelSort, a sorting algorithm that is correct, terminates, and yet cannot be proven to terminate within Peano Arithmetic. Our algorithm achieves optimal pessimal complexity of $\Theta(f_{\varepsilon_0}(n))$, placing it firmly outside the realm of primitive recursive functions.

The gap between "terminates" and "usably terminates" is unbounded. One might naively think that if you can prove something terminates, you can extract some reasonable bound on when. GödelSort is a demonstration that this is false not just by a large constant factor, but by more than any primitive recursive function. The proof-theoretic machinery required to verify termination tells you nothing useful about wall clock time.

We hope this work inspires further research into pessimal algorithms, unprovable termination, and the boundaries between logic and computation. In formal methods, we often want to prove programs correct and terminating. GödelSort satisfies both conditions and is utterly useless. This is a pointed reminder that specifications need to capture what you actually care about. If you didn't ask for a complexity bound, you didn't get one.

As Erdős would say: this proof is straight from The Book.[1]

Acknowledgments

We thank Ernst Zermelo for the well-ordering, Abraham Fraenkel for the replacement, and the Axiom of Choice for being there when we needed it. Author order was determined by the Well-Ordering Theorem. We cannot construct it explicitly but are assured it exists.

We are grateful to Giuseppe Peano, whose arithmetic was insufficiently powerful to verify our termination claims. This is a feature, not a bug.

Communication with the deceased coauthors was conducted via séance [2]. The board spelled out "MY BRAIN IS OPEN" (Erdős) and "ABER WAS BEDEUTET DAS" (Gödel). We interpret this as enthusiastic consent.

Finally, we dedicate this paper to the Continuum Hypothesis, which has nothing to do with our work but feels left out.

References

[1] Andrei Broder and Jorge Stolfi. Pessimal algorithms and simplexity analysis. *SIGACT News*, 16(3):49–53, September 1984. doi:10.1145/990534.990536.

[1] The Book of algorithms god keeps, containing only the worst possible solutions.

[2] Paul Erdős. Personal communication. Via séance, January 2026. The board spelled "MY BRAIN IS OPEN" followed by "THIS PROOF IS FROM THE BOOK." Communication facilitated by standard Ouija protocol with two independent witnesses.

[3] R. L. Goodstein. On the restricted ordinal theorem. *Journal of Symbolic Logic*, 9(2):33–41, 1944. doi:10.2307/2268019.

[4] Hermann Gruber, Markus Holzer, and Oliver Ruepp. Sorting the slow way: An analysis of perversely awful randomized sorting algorithms. In *Fun with Algorithms*, volume 4475 of *Lecture Notes in Computer Science*, pages 183–197, Berlin, Heidelberg, 2007. Springer. doi:10.1007/978-3-540-72914-3_17.

[5] C. A. R. Hoare. Algorithm 64 — quicksort. *Communications of the ACM*, 4(7):321, 1961. doi:10.1145/366622.366644.

[6] L. Kirby and J. Paris. Accessible independence results for peano arithmetic. *Bulletin of the London Mathematical Society*, 14(4):285–293, 1982. doi:10.1112/blms/14.4.285.

[7] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, MA, 3 edition, 1997. Section 5.2.5: Sorting by Distribution, pp. 168–179.

[8] J. W. J. Williams. Algorithm 232 — heapsort. *Communications of the ACM*, 7(6):347–348, 1964. doi:10.1145/512274.512284.

A Appendix: Implementation

We provide the complete implementation of GÖDELSORT in Python. This code is correct, terminates, and we cannot prove it terminates in PA.

```
"""
GÖDELSORT: A Provably Terminating Sorting Algorithm
           With Unprovable Runtime
"""

from typing import List, Tuple

def to_hereditary_base(n: int, base: int) -> List[Tuple[int, any]]:
    """
    Convert n to hereditary base notation.

    In hereditary base b, we write n as a sum of powers of b,
    where the exponents are ALSO written in hereditary base b.
    """
    if n == 0:
        return []

    result = []
    power = 0
```

```
    while n > 0:
        coeff = n % base
        if coeff > 0:
            exp_hereditary = to_hereditary_base(power, base)
            result.append((coeff, exp_hereditary))
        n //= base
        power += 1

    return result


def from_hereditary_base(rep: List[Tuple[int, any]], base: int) -> int:
    if not rep:
        return 0

    total = 0
    for coeff, exp_rep in rep:
        exp_value = from_hereditary_base(exp_rep, base)
        total += coeff * (base ** exp_value)

    return total


def bump_base(rep: List[Tuple[int, any]],
              old_base: int, new_base: int) -> List[Tuple[int, any]]:
    """
    Replace all occurrences of old_base with new_base.
    This is the operation that makes Goodstein sequences
    grow enormously before eventually reaching zero.
    """
    if not rep:
        return []

    result = []
    for coeff, exp_rep in rep:
        new_exp = bump_base(exp_rep, old_base, new_base)
        result.append((coeff, new_exp))

    return result


def goodstein_step(n: int, base: int) -> Tuple[int, int]:
    """
    Perform one step of the Goodstein sequence
    1. Write n in hereditary base `base`
    2. Bump the base to `base + 1`
    3. Subtract 1
    """
```

```
    rep = to_hereditary_base(n, base)
    bumped = bump_base(rep, base, base + 1)
    new_n = from_hereditary_base(bumped, base + 1) - 1
    return new_n, base + 1


def goodstein_sequence(n: int) -> None:
    """
    Run the Goodstein sequence starting from n.

    Goodstein's Theorem says this ALWAYS terminates.
    Kirby-Paris (1982) PA cannot prove this terminates.
    """
    base = 2
    current = n

    while current > 0:
        current, base = goodstein_step(current, base)


def godelsort(arr: List) -> List:
    """
    GÖDELSORT: slowest correct sorting algorithm.

    Correctness: Trivial. We sort at the end.
    Termination: By Goodstein's theorem (requires epsilon_0 induction).
    Runtime: f_{epsilon_0}(n), faster than no primitive recursive function.

    Note: For n >= 4, you may wish to prepare snacks.
          For n >= 5, please ensure the heat death of the
          universe is accounted for in your project timeline.
    """
    goodstein_sequence(len(arr))
    return sorted(arr)


# Demo
if __name__ == "__main__":
    print(godelsort([3, 1, 2]))  # Works! Returns [1, 2, 3]
    # print(godelsort([4, 2, 3, 1]))  # Do not run
```

B Benchmark Trace for $n = 4$

```
Step 0: m = 4, base = 2
Step 1: m = 26, base = 3
Step 2: m = 41, base = 4
Step 3: m = 60, base = 5
```

```
Step 4: m = 83, base = 6
Step 5: m = 109, base = 7
Step 6: m = 139, base = 8
Step 7: m = 173, base = 9
Step 8: m = 211, base = 10
Step 9: m = 253, base = 11
...
Step 1000000: m = 10896842496, base = 1000002
[BENCHMARK CONTINUES]
```